MANUAL DE DESPLIEGUE — AGROGESTIÓN

Índice

1. Introducción
2. Requisitos previos
3. Obtener el código fuente
4. Configurar Supabase (Backend)
    - 4.1. Crear proyecto en Supabase
    - 4.2. Crear las tablas de la base de datos
    - 4.3. Configurar políticas RLS
    - 4.4. Obtener credenciales de conexión
5. Configurar el entorno local
    - 5.1. Instalar Node.js
    - 5.2. Instalar dependencias del proyecto
    - 5.3. Configurar variables de entorno
6. Ejecutar la aplicación en modo desarrollo
7. Construir la aplicación para producción
8. Desplegar en un servidor
    - 8.1. Despliegue en Vercel (recomendado)
    - 8.2. Despliegue en Netlify
    - 8.3. Despliegue en servidor propio (Apache/Nginx)
9. Verificación del despliegue
10. Solución de problemas comunes

---

1. Introducción

Este manual describe paso a paso todo lo necesario para poner en marcha la aplicación Agrogestión, tanto en un entorno de desarrollo local como en un servidor de producción.

Está dirigido a desarrolladores o técnicos encargados de instalar, configurar y desplegar la aplicación.

---

2. Requisitos previos

Antes de comenzar, asegúrate de tener instalado lo siguiente:

Software	Versión mínima	Descarga
Node.js	18.x (recomendado 20.x LTS)	https://nodejs.org/
npm	9.x (incluido con Node.js)	(se instala con Node.js)
Git	2.x	https://git-scm.com/
Navegador	Chrome 90+, Firefox 90+	(cualquier navegador moderno)

Verificar las versiones instaladas:

Abre un terminal (CMD, PowerShell o Bash) y ejecuta:

```
node --version
# Debe mostrar v18.x.x o superior

npm --version
# Debe mostrar 9.x.x o superior

git --version
# Debe mostrar git version 2.x.x
```

Si alguno de los comandos no es reconocido, debes instalar el software correspondiente desde los enlaces indicados.

---

3. Obtener el código fuente

Opción A: Clonar desde GitHub

```
git clone https://github.com/Juanfrina/Agrogestion.Web.git
cd Agrogestion.Web
cd AGROGESTION
```

Opción B: Descargar como ZIP

1. Ir a https://github.com/Juanfrina/Agrogestion.Web
2. Pulsar el botón verde **"Code"** → **"Download ZIP"**.
3. Descomprimir el archivo en la ubicación deseada.
4. Abrir un terminal y navegar hasta la carpeta `AGROGESTION/` dentro del proyecto.

---

4. Configurar Supabase (Backend)

4.1. Crear proyecto en Supabase

1. Accede a https://supabase.com y crea una cuenta (puedes usar GitHub para registrarte).
2. En el dashboard, pulsa **"New Project"**.
3. Configura el proyecto:
   - **Nombre del proyecto:** agrogestion
   - **Contraseña de la base de datos:** (elige una contraseña segura y guárdala)
   - **Región:** Elige la más cercana a tu ubicación (ej: West EU - Ireland)
4. Pulsa **"Create new project"** y espera a que se aprovisione (1-2 minutos).

4.2. Crear las tablas de la base de datos

Una vez creado el proyecto, ve a **SQL Editor** en el menú lateral izquierdo de Supabase y ejecuta el siguiente script SQL:

```sql
-- ======================================================
-- SCRIPT DE CREACIÓN DE TABLAS — AGROGESTIÓN
-- ======================================================

-- Tabla de roles
CREATE TABLE IF NOT EXISTS rol (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

-- Insertar roles por defecto
INSERT INTO rol (nombre) VALUES ('ADMIN'), ('GERENTE'), ('CAPATAZ'),
('TRABAJADOR');

-- Tabla de usuarios (perfil extendido)
CREATE TABLE IF NOT EXISTS usuario (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    nombre VARCHAR(100) NOT NULL,
    apellidos VARCHAR(150) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    id_rol INTEGER REFERENCES rol(id),
    activo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Tabla de terrenos
CREATE TABLE IF NOT EXISTS terreno (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    ubicacion VARCHAR(255),
    superficie DECIMAL(10,2),
    id_gerente UUID REFERENCES usuario(id),
    activo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Tabla de estados de tarea
CREATE TABLE IF NOT EXISTS estados_tarea (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion TEXT
);

-- Insertar estados por defecto
INSERT INTO estados_tarea (nombre, descripcion) VALUES
    ('PENDIENTE', 'La tarea ha sido creada pero no ha comenzado.'),
    ('EN_PROGRESO', 'La tarea está siendo ejecutada por el capataz.'),
    ('COMPLETADA', 'La tarea ha sido finalizada correctamente.');

-- Tabla de tareas
CREATE TABLE IF NOT EXISTS tarea (
    id SERIAL PRIMARY KEY,
```

```
    descripcion TEXT NOT NULL,
    id_terreno INTEGER REFERENCES terreno(id),
    id_gerente UUID REFERENCES usuario(id),
    id_capataz UUID REFERENCES usuario(id),
    id_estado INTEGER REFERENCES estados_tarea(id) DEFAULT 1,
    fecha_creacion TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Tabla relación gerente-capataz (N:M)
CREATE TABLE IF NOT EXISTS gerente_capataz (
    id_gerente UUID REFERENCES usuario(id) ON DELETE CASCADE,
    id_capataz UUID REFERENCES usuario(id) ON DELETE CASCADE,
    PRIMARY KEY (id_gerente, id_capataz)
);

-- Tabla relación tarea-trabajador (N:M)
CREATE TABLE IF NOT EXISTS tarea_trabajador (
    id_tarea INTEGER REFERENCES tarea(id) ON DELETE CASCADE,
    id_trabajador UUID REFERENCES usuario(id) ON DELETE CASCADE,
    PRIMARY KEY (id_tarea, id_trabajador)
);

-- Tabla relación capataz-trabajador (N:M)
CREATE TABLE IF NOT EXISTS capataz_trabajador (
    id_capataz UUID REFERENCES usuario(id) ON DELETE CASCADE,
    id_trabajador UUID REFERENCES usuario(id) ON DELETE CASCADE,
    PRIMARY KEY (id_capataz, id_trabajador)
);
```

Pulsa **"Run"** para ejecutar el script. Deberías ver el mensaje "Success" sin errores.

4.3. Configurar políticas RLS

En el menú lateral de Supabase, ve a **Authentication** → **Policies** (o en cada tabla desde **Table Editor**).

Habilitar RLS en todas las tablas:

```
-- Habilitar RLS
ALTER TABLE usuario ENABLE ROW LEVEL SECURITY;
ALTER TABLE terreno ENABLE ROW LEVEL SECURITY;
ALTER TABLE tarea ENABLE ROW LEVEL SECURITY;
ALTER TABLE gerente_capataz ENABLE ROW LEVEL SECURITY;
ALTER TABLE tarea_trabajador ENABLE ROW LEVEL SECURITY;
ALTER TABLE capataz_trabajador ENABLE ROW LEVEL SECURITY;
ALTER TABLE rol ENABLE ROW LEVEL SECURITY;
ALTER TABLE estados_tarea ENABLE ROW LEVEL SECURITY;

-- Política: Todos los usuarios autenticados pueden leer roles y estados
CREATE POLICY "Roles visibles para usuarios autenticados"
    ON rol FOR SELECT
    TO authenticated
    USING (true);
```

```sql
CREATE POLICY "Estados visibles para usuarios autenticados"
    ON estados_tarea FOR SELECT
    TO authenticated
    USING (true);

-- Política: Cada usuario puede ver su propio perfil
CREATE POLICY "Usuario puede ver su propio perfil"
    ON usuario FOR SELECT
    TO authenticated
    USING (auth.uid() = id);

-- Política: Admin puede ver todos los usuarios
CREATE POLICY "Admin puede ver todos los usuarios"
    ON usuario FOR SELECT
    TO authenticated
    USING (
        EXISTS (
            SELECT 1 FROM usuario u
            WHERE u.id = auth.uid()
            AND u.id_rol = (SELECT id FROM rol WHERE nombre = 'ADMIN')
        )
    );

-- Política: Admin puede actualizar usuarios
CREATE POLICY "Admin puede actualizar usuarios"
    ON usuario FOR UPDATE
    TO authenticated
    USING (
        EXISTS (
            SELECT 1 FROM usuario u
            WHERE u.id = auth.uid()
            AND u.id_rol = (SELECT id FROM rol WHERE nombre = 'ADMIN')
        )
    );

-- Política: Gerente puede ver sus terrenos
CREATE POLICY "Gerente puede ver sus terrenos"
    ON terreno FOR SELECT
    TO authenticated
    USING (id_gerente = auth.uid());

-- Política: Gerente puede crear terrenos
CREATE POLICY "Gerente puede crear terrenos"
    ON terreno FOR INSERT
    TO authenticated
    WITH CHECK (id_gerente = auth.uid());

-- Política: Gerente puede actualizar sus terrenos
CREATE POLICY "Gerente puede actualizar sus terrenos"
    ON terreno FOR UPDATE
    TO authenticated
    USING (id_gerente = auth.uid());
```

```sql
-- Política: Gerente puede ver las tareas que ha creado
CREATE POLICY "Gerente puede ver sus tareas"
    ON tarea FOR SELECT
    TO authenticated
    USING (id_gerente = auth.uid());

-- Política: Capataz puede ver sus tareas asignadas
CREATE POLICY "Capataz puede ver sus tareas"
    ON tarea FOR SELECT
    TO authenticated
    USING (id_capataz = auth.uid());

-- Política: Gerente puede crear tareas
CREATE POLICY "Gerente puede crear tareas"
    ON tarea FOR INSERT
    TO authenticated
    WITH CHECK (id_gerente = auth.uid());

-- Política: Capataz puede actualizar estado de sus tareas
CREATE POLICY "Capataz puede actualizar sus tareas"
    ON tarea FOR UPDATE
    TO authenticated
    USING (id_capataz = auth.uid());
```

4.4. Obtener credenciales de conexión

1. En Supabase, ve a **Settings** → **API** (en el menú lateral).
2. Copia los siguientes valores:
   - **Project URL** (ej: `https://xxxxx.supabase.co`)
   - **anon public key** (una cadena larga que empieza por `eyJ...`)
3. Estos valores se usarán en el siguiente paso.

---

5. Configurar el entorno local

5.1. Instalar Node.js

1. Descarga Node.js desde https://nodejs.org (versión LTS recomendada).
2. Ejecuta el instalador y sigue los pasos (acepta los valores por defecto).
3. Reinicia el terminal.
4. Verifica la instalación:

```
node --version
npm --version
```

5.2. Instalar dependencias del proyecto

Desde el directorio raíz del proyecto (`AGROGESTION/`), ejecuta:

```
npm install
```

Este comando descargará todas las dependencias definidas en `package.json`. Puede tardar 1-3 minutos dependiendo de la conexión a Internet.

Dependencias principales que se instalarán:

Paquete	Función
react, react-dom	Librería de interfaz de usuario.
@supabase/supabase-js	Cliente para comunicar con Supabase.
tailwindcss	Framework CSS de utilidades.
@tailwindcss/vite	Plugin de TailwindCSS para Vite.
react-router-dom	Enrutamiento y navegación SPA.
i18next, react-i18next	Internacionalización (español/inglés).
zustand	Gestión de estado global ligera.
dotenv	Gestión de variables de entorno.
vite	Herramienta de build y servidor de desarrollo.
typescript	Compilador TypeScript.

5.3. Configurar variables de entorno

Crea un archivo llamado `.env` en la raíz de la carpeta `AGROGESTION/` con el siguiente contenido:

```
VITE_SUPABASE_URL=https://TU_PROYECTO.supabase.co
VITE_SUPABASE_ANON_KEY=TU_CLAVE_ANON_PUBLICA
```

Sustituye los valores por los que copiaste en el paso 4.4.

> ⚠ **IMPORTANTE:** El archivo `.env` contiene información sensible. NO lo subas a repositorios públicos. Ya está incluido en `.gitignore`.

---

6. Ejecutar la aplicación en modo desarrollo

Desde la carpeta `AGROGESTION/`, ejecuta:

```
npm run dev
```

Verás un mensaje similar a:

```
VITE v7.2.4  ready in 300ms

➜  Local:   http://localhost:5173/
➜  Network: http://192.168.x.x:5173/
➜  press h + enter to show help
```

Abre tu navegador y accede a:

http://localhost:5173

La aplicación se recargará automáticamente (Hot Module Replacement) cada vez que guardes cambios en el código.

Para detener el servidor de desarrollo, pulsa `Ctrl + C` en el terminal.

---

7. Construir la aplicación para producción

Para generar los archivos optimizados para producción:

```
npm run build
```

Este comando:

1. Compila el código TypeScript.
2. Empaqueta y minifica todos los archivos (HTML, CSS, JavaScript).
3. Genera la carpeta `dist/` con los archivos listos para subir a un servidor.

Para previsualizar el build antes de subir:

```
npm run preview
```

Se abrirá un servidor local en `http://localhost:4173` con la versión de producción.

---

8. Desplegar en un servidor

8.1. Despliegue en Vercel (recomendado)

Vercel es la plataforma recomendada para proyectos Vite/React.

Pasos:

1. Crea una cuenta en https://vercel.com (puedes usar GitHub).
2. Pulsa **"Add New Project"**.
3. Selecciona el repositorio **Juanfrina/Agrogestion.Web** desde GitHub.
4. Configura el proyecto:

    - **Framework Preset:** Vite
    - **Root Directory:** `AGROGESTION`
    - **Build Command:** `npm run build`
    - **Output Directory:** `dist`
5. Añade las **variables de entorno**:
    - Clave: `VITE_SUPABASE_URL` → Valor: tu URL de Supabase
    - Clave: `VITE_SUPABASE_ANON_KEY` → Valor: tu anon key
6. Pulsa **"Deploy"**.
7. En 1-2 minutos tendrás la URL de producción (ej: `https://agrogestion.vercel.app`).

Despliegues posteriores: Cada push a la rama `main` en GitHub desplegará automáticamente la nueva versión.

8.2. Despliegue en Netlify

1. Crea una cuenta en https://netlify.com.
2. Pulsa **"Add new site"** → **"Import an existing project"**.
3. Conecta tu cuenta de GitHub y selecciona el repositorio.
4. Configura:
    - **Base directory:** `AGROGESTION`
    - **Build command:** `npm run build`
    - **Publish directory:** `AGROGESTION/dist`
5. Añade las variables de entorno en **Site settings** → **Environment variables**.
6. Pulsa **"Deploy site"**.

8.3. Despliegue en servidor propio (Apache/Nginx)

Paso 1: Construir la aplicación

```
cd AGROGESTION
npm install
npm run build
```

Paso 2: Subir los archivos

Copia el contenido de la carpeta `dist/` al directorio raíz del servidor web:

- **Apache:** `/var/www/html/agrogestion/`
- **Nginx:** `/usr/share/nginx/html/agrogestion/`

Puedes usar FTP, SCP o cualquier método de transferencia:

```
scp -r dist/* usuario@servidor:/var/www/html/agrogestion/
```

Paso 3: Configurar redirección SPA

Como Agrogestión es una SPA (Single Page Application), todas las rutas deben redirigirse a `index.html`.

Apache — Crea un archivo `.htaccess` en la carpeta del proyecto:

```
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteBase /
    RewriteRule ^index\.html$ - [L]
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule . /index.html [L]
</IfModule>
```

Nginx — Añade al bloque `server`:

```
server {
    listen 80;
    server_name tu-dominio.com;
    root /usr/share/nginx/html/agrogestion;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Reinicia el servidor web tras los cambios:

```
# Apache
sudo systemctl restart apache2

# Nginx
sudo systemctl restart nginx
```

---

9. Verificación del despliegue

Una vez desplegada la aplicación, verifica lo siguiente:

Verificación	Cómo comprobar
La página carga correctamente	Accede a la URL y verifica que se muestra la landing page.
La conexión con Supabase funciona	Intenta registrarte o iniciar sesión.
Las rutas funcionan correctamente	Navega entre páginas y recarga (F5) en cada una.
El diseño responsive es correcto	Prueba en móvil o redimensiona el navegador.

Verificación	Cómo comprobar
No hay errores en la consola del navegador	Abre DevTools (F12) y revisa la pestaña Console.
Las políticas RLS funcionan	Inicia sesión con distintos roles y verifica los datos visibles.

10. Solución de problemas comunes

Error: "npm: command not found"

- **Causa:** Node.js no está instalado o no está en el PATH.
- **Solución:** Instala Node.js desde https://nodejs.org y reinicia el terminal.

Error: "VITE_SUPABASE_URL is undefined"

- **Causa:** El archivo `.env` no existe o las variables no tienen el prefijo `VITE_`.
- **Solución:** Verifica que el archivo `.env` existe en la carpeta `AGROGESTION/` y que las variables empiezan por `VITE_`.

Error: "Failed to fetch" al hacer login

- **Causa:** La URL de Supabase o la anon key son incorrectas.
- **Solución:** Verifica los valores en `.env` comparándolos con los de Settings → API en Supabase.

Error: "relation 'usuario' does not exist"

- **Causa:** Las tablas no se han creado en Supabase.
- **Solución:** Ejecuta el script SQL del paso 4.2 en el SQL Editor de Supabase.

La página se queda en blanco

- **Causa:** Error de JavaScript. Puede ser un problema de rutas o de build.
- **Solución:** Abre la consola del navegador (F12 → Console) para ver el error específico.

Error 404 al recargar una ruta (producción)

- **Causa:** El servidor no está configurado para redirigir a `index.html`.
- **Solución:** Configura la redirección SPA según el paso 8.3.

Error de CORS

- **Causa:** El dominio de producción no está en la lista de orígenes permitidos de Supabase.
- **Solución:** En Supabase → Settings → API → añade tu dominio en "Additional Redirect URLs".

npm install falla con errores de permisos

- **Causa:** Problemas de permisos en Windows o Linux.
- **Solución Windows:** Ejecuta el terminal como Administrador.
- **Solución Linux/Mac:** No uses `sudo npm install`. Configura npm para que no requiera permisos de root.

Resumen de comandos

Acción	Comando
Clonar repositorio	`git clone https://github.com/Juanfrina/Agrogestion.Web.git`
Entrar en la carpeta	`cd Agrogestion.Web/AGROGESTION`
Instalar dependencias	`npm install`
Ejecutar en desarrollo	`npm run dev`
Construir para producción	`npm run build`
Previsualizar build	`npm run preview`
Verificar versión de Node.js	`node --version`
Verificar versión de npm	`npm --version`

Agrogestión — Manual de Despliegue Autor: Juan Francisco Hurtado Pérez Ciclo: CFGS Desarrollo de Aplicaciones Web (DAW) Centro: IES Albarregas – Mérida (España)